

Person B — Platform & Agent Engineer

Codebase PRD — Patient Risk Intelligence MCP Platform

Sprint: June 25 — July 9, 2026 (2 people, 2 weeks)

Companion to: Person A PRD · Combined Sprint PRD · Full Patient Risk Intelligence PRD v1.0

1. Your Mission

You own the road and the traveler: identity, gateway, the control-plane registry, the onboarding agent that proposes new servers, the runtime agent that answers clinical questions, and the frontend a clinician actually sees. Person A's 4 servers are useless without your agent to call them and your gateway to secure them.

Your riskiest dependency is a token shape agreement, not a code dependency — lock down the JWT claims with Person A before either of you writes auth code (Section 6). Everything else, you can build against a stub server from day 2 onward without waiting for Person A's real servers.

2. Scope — What You're Building

- Keycloak: 3 roles (clinical-viewer, physician, case-manager), OAuth 2.1 token issuing.
- Kong Gateway: Layer-1 security — token validation, tiered rate limits per role, routing, 401/429 handling.
- registry-db (Postgres): the 12-table control-plane source of truth (products, mcp_servers, connectors, tool_specs, rbac_mappings, deployment_environments, gateway_routes, api_registry, agent_integrations, audit_events, health_checks, schema_snapshots).
- Onboarding Agent: discover → suggest tools → draft RBAC → assemble YAML blueprint. Output is a file only — it never deploys anything.
- Runtime Agent (LangGraph Host): holds 4 MCP Clients, attaches tokens, calls tools, fuses 4 sources into one cited answer.
- Frontend: CopilotKit chat (+ human-in-the-loop approval card for onboarding), Tremor dashboard (KPIs, registry status, RBAC breakdown).
- Frontend login flow (gap fix): NextAuth.js configured against Keycloak's OIDC endpoint — closes a gap where there was no documented way for a user's token to actually reach the Runtime Agent.
- registry-api access control (gap fix): require a valid, non-anonymous Keycloak token on every call — closes a gap where operational/audit data was reachable with no auth at all.
- Purpose-of-access enum support: the chat UI's reason-for-query selector must offer exactly the 5 values Person A's audit.py validates against — see Section 6.3.
- docker-compose.platform.yml — your half of the infrastructure file (see Section 7).

3. Out of Scope for This Sprint — Not Your Job

Item	Why cut / what replaces it
Backstage real software catalog	Query registry-db directly with SQL/a small script instead of standing up Backstage.
GLPI / real change-governance tool	Document the change-gate concept in the PRD; don't build a Stage/Prod gate this sprint (single environment only).
Full 13-stage CI/CD pipeline	GitHub Actions: build + test on push is enough.

Auto-generated 20-item documentation	This PRD is the documentation for this sprint.
PHI access anomaly Grafana panel	Stub only if time allows after Jul 7; not required for the demo.

4. Your Day-by-Day Schedule

Date	Your task
Thu Jun 25	Keycloak up (3 roles). Kong up. docker-compose.platform.yml skeleton.
Fri Jun 26	Wire Kong↔Keycloak token validation. Stand up registry-db (12 tables). Receive Person A's stub server — start building against it immediately.
Mon Jun 29	Build Onboarding Agent: discover → suggest → draft RBAC → assemble blueprint (against the stub schema).
Tue Jun 30	Build Runtime Agent skeleton: Host + Client code, against the stub server.
Wed Jul 1	Integrate Runtime Agent with real vitals_trends + labs_diagnoses (Person A should have these by now).
Thu Jul 2	Integrate medications_interactions; finalize 3-source fusion logic with citations.
Fri Jul 3	CopilotKit chat wired to agent + NextAuth.js login against Keycloak. OpenTelemetry/Jaeger across 3 servers. CHECKPOINT — confirm with Person A whether server #4 is on track.
Mon Jul 6	Tremor dashboard skeleton (KPIs, registry status from registry-db, RBAC breakdown). Add the non-anonymous token check to registry-api.
Tue Jul 7	Integrate 4th MCP Client (clinical_notes_search) if Person A shipped it; finalize citation-based answer synthesis.
Wed Jul 8	Rate-limit (429) and expired-token (401) demo configs in Kong. Record a full backup demo video.
Thu Jul 9	Final fixes. Live demo support.

If Person A drops clinical_notes_search at the Jul 3 checkpoint: simplify your Tue Jul 7 task to hardening the 3-source fusion instead of integrating a 4th client. Don't build agent logic for a server that won't exist.

5. Components You Own — Full Detail

5.1 Identity — Keycloak

3 roles, deliberately different from the Batch Release platform's roles, proving RBAC is blueprint-driven, not hardcoded:

- clinical-viewer (nurse) — vitals, labs only
- physician — all 4 domains
- case-manager — clinical notes only

Tokens must carry: sub, oid (used as audit 'who'), groups[], scp (space-separated scopes). See Section 6 — this exact shape is a contract with Person A.

5.2 Gateway — Kong

- OAuth 2.1 token validation against Keycloak's JWKS endpoint
- Tiered rate limits per role (physician gets a higher quota than case-manager)
- Routing: /mcp/{domain}/{environment} → correct server
- 401 on invalid/expired token, 429 on over-quota — both need a live demo on Jul 8

5.3 registry-db (Postgres, 12 tables)

The control-plane source of truth. Build the schema first (Jun 26), then a small read API so the Onboarding Agent and Tremor dashboard can query it. Tables: products, mcp_servers, connectors, tool_specs, rbac_mappings, deployment_environments, gateway_routes, api_registry, agent_integrations, audit_events, health_checks, schema_snapshots.

5.4 Onboarding Agent (build-time)

One process, internal stages — not 4 separate deployed agents:

- discover.py — reads the target source's schema (FHIR shape, not just SQL columns)
- suggest_tools.py — LLM call turning schema into readable tool descriptions
- draft_rbac.py — applies the role matrix, drafts per-tool scopes
- assemble_blueprint.py — writes the YAML file. This agent never deploys anything — a human approves the file, then a separate (out-of-scope-this-sprint) pipeline would build from it.

5.5 Runtime Agent (LangGraph Host)

Holds 4 MCP Clients (one per server — see contract in Section 6 for exact tool names). On a question like “What is this patient's overall risk picture?”, calls all relevant tools in parallel, then synthesizes one answer that cites which server each fact came from — e.g. “NEWS2 score 6 (vitals_trends); 3 high-risk medication interactions (medications_interactions).”

5.6 Frontend

- Chat page (CopilotKit): hosts the Runtime Agent conversation; renders the human-in-the-loop approval card during onboarding (useHumanInTheLoop); includes a reason-for-query selector (the purpose_of_access enum, see Section 6.3).
- Dashboard page (Tremor): KPIs (questions asked, denials), registry status pulled live from registry-db, RBAC allow/deny breakdown by role.

5.6.1 Login Flow (gap fix)

Without this, there was no documented way for a user's identity to reach the Runtime Agent at all. NextAuth.js, configured with Keycloak as an OIDC provider (frontend/app/api/auth/[...nextauth]/route.ts), handles the actual login. On success, NextAuth holds a session containing the Keycloak-issued access token (the same shape as Section 6.1). Every call from the chat page to the Runtime Agent's /ask endpoint attaches this token as a

Bearer header — that token is what the Runtime Agent then forwards to each MCP server through Kong.

5.6.2 registry-api Access Control (gap fix)

registry-api previously had zero authentication — anyone on the network could read audit_events and other operational tables. Fix: require any valid (non-expired, correctly signed) Keycloak token on every request — not full per-role RBAC, just “not anonymous.”
backend/registry/auth.py: require_valid_token(request) -> dict, reusing the same JWKS verification as the MCP servers' auth.py.

6. The Contract You Must Honor

This is the same contract Person A's PRD references — it must match exactly, or integration breaks.

6.1 Token shape you issue (Person A reads this)

```
JWT claims: { "sub": "<user id>", "oid": "<object id, used in audit 'who'>",
  "groups": ["grp-clinical-viewer" | "grp-physician" | "grp-case-manager"],
  "scp": "mcp.vitals.read mcp.labs.read ..." } (space-separated scopes)
```

6.2 Tool names + scopes you must call exactly

Server	Tools	Scope
vitals_trends	get_vitals_trend, compute_news2_score, list_abnormal_vitals	mcp.vitals.read
labs_diagnoses	get_lab_trend, get_active_diagnoses, get_diagnosis_history	mcp.labs.read
medications_interactions	get_active_medications, check_drug_interactions, get_polypharmacy_risk	mcp.meds.read
clinical_notes_search	semantic_search_notes, get_recent_notes, get_notes_by_type	mcp.notes.read

6.3 purpose_of_access Enum (gap fix — fixed vocabulary, not free text)

Your chat UI's reason-for-query selector must offer exactly these 5 values, matching what Person A's audit.py validates against:

- deterioration_review · medication_reconciliation · discharge_planning · care_coordination · routine_review (default)

6.4 What you depend on from Person A, and when

- Day 2 (Jun 26): a stub server — hardcoded JSON, real tool names, real route. You start building the agent against this immediately.
- Day 7 (Jul 3) checkpoint: confirmation of whether all 4 real servers will land, or only 3.
- Denial responses always arrive as { "error": { "code": "forbidden", "reason": "..." } } — your chat UI renders `reason` directly.

7. Your docker-compose.platform.yml

You own this file; Person A owns docker-compose.data.yml. Merge into one file together on Jul 8.

- keycloak, kong
- registry-db
- frontend (Next.js: chat + dashboard)
- The agent process(es), once built

8. Your Directory Structure

```

backend/
├── onboarding_agent/
│   ├── discover.py
│   ├── suggest_tools.py
│   ├── draft_rbac.py
│   └── assemble_blueprint.py
├── registry/
│   ├── registry_api.py
│   └── auth.py                # NEW – non-anonymous token check
└──

agent/
├── runtime_agent.py          # LangGraph Host, 4 MCP Clients
└──

frontend/
├── app/
│   ├── chat/page.tsx        # CopilotKit chat + approval card
│   ├── dashboard/page.tsx   # Tremor
│   └── api/auth/[...nextauth]/route.ts # NEW – Keycloak login
├── components/
│   ├── ApprovalCard.tsx
│   └── RegistryTable.tsx
└──

infra/
├── kong/kong.yml
├── keycloak/realm-export.json
└── postgres/init-registry-db.sql

```

9. Tech Stack You'll Use

Component	Choice	Version
Identity	Keycloak	26+
Gateway	Kong Gateway OSS	3.14
Control-plane DB	PostgreSQL	16
Agent orchestration	LangGraph	—
Frontend framework	Next.js + React + Tailwind	—
Chat / human-in-the-loop	CopilotKit	useHumanInTheLoop, useInterrupt
Dashboard	Tremor	React + Tailwind + Radix
MCP SDK (for the agent's clients)	mcp	v1.28.x — must match Person A's pin exactly

10. Acceptance Criteria — Definition of Done

- A physician token successfully calls all 4 servers; a clinical-viewer token is correctly denied on medications_interactions and clinical_notes_search.
- The Runtime Agent produces one synthesized answer citing which server each fact came from.
- A repeated call beyond a role's Kong quota returns 429, visibly, in the demo.
- An expired token returns a clean 401, not a crash.
- The Tremor dashboard's registry status reflects registry-db's actual contents live.
- The onboarding agent produces a valid YAML blueprint from a real schema discovery pass.
- A user can log in via NextAuth/Keycloak and the resulting token successfully authenticates their first chat question.
- A request to registry-api with no token (or a garbage token) is rejected; a request with a valid token succeeds.
- Selecting a purpose_of_access reason in the chat UI before asking a question results in that exact value appearing in the audit record.

11. Daily Sync — What to Report

15 minutes, every day, with Person A. Report: what you finished yesterday, what's blocking you, and confirm the contract in Section 6 hasn't silently drifted on either side.